PRACTICAL GUIDE / 18 OCTOBER 2026

Website & Web App

Planning Toolkit

Plan clearly.
Build in small steps.

Six reusable documents for a solo developer working with an AI coding agent. Use this visual guide with the six editable Markdown templates.

What & why → **How it works** → **How to verify**

These are connected decisions, not a mandatory writing order.

Document	Its main question	Pages
01 Product requirements	What should the product do, and why?	3-4
02 Technical requirements	How will the system work?	5-6
03 App flow	How does the user complete a task?	7-8
04 Design brief	How should the interface look and respond?	9-10
05 Data and access	What data exists, and who can access it?	11-12
06 Implementation plan	What gets built next, and how is it checked?	13-14
Connected example	Follow one feature through all six documents.	15-16

Use only what helps
Fill the core decisions for the next feature. Keep conditional sections only when relevant. A static portfolio does not need an invented backend.

Examples are hypothetical. Blank fields are intentional. No test results, professional certification, or new JEV results are claimed.

START HERE

How to use the toolkit

One home for each decision; enough detail for the next feature.

Define the need → **Resolve key choices** → **Build a small slice** → **Check & update**

If a decision is uncertain, use a small experiment before dependent implementation.

Document	Owns this decision
PRD.md	User needs, scope, priorities, required behavior, business rules.
TECHNICAL_REQUIREMENTS.md	Architecture, integration contracts, engineering constraints and enforcement boundaries.
APP_FLOW.md	Navigation, interaction order, validation timing, states and recovery.
DESIGN.md	Visual presentation, components, content style and accessibility presentation.
BACKEND_SCHEMA.md	Data definitions, integrity, ownership, permissions and lifecycle.
IMPLEMENTATION_PLAN.md	Work order, dependencies, task status and verification evidence.

Shared header: Project [name] / Owner [role] / Document review [Draft or Reviewed for current scope] / Last substantive review [date] / Current scope [feature].

- **Essential (core)** means answer briefly for the next feature. **Optional (conditional)** means keep only if relevant.
- **Deferred** means record the question, owner and trigger. [Brackets] mean fill in real project information.
- Link to the owning document. Resolve conflicts there before dependent work; continue unaffected work.

Keep the records separate: document review describes the specification; ADR status describes a decision; task status describes progress; check status describes a test result.

Further reading: Atlassian PRDs / GitLab design workflow / AWS decision records
NN/G wireflows / OWASP authorization / W3C accessibility

> **Choose the minimum useful depth**
> Static portfolio: brief scope, flow, design, hosting, data-handling statement and named checks. Add database, permissions, integrations or migrations only when the feature needs them. The ZIP includes six templates, README.md and WORKED_EXAMPLE.md.

01 / TEMPLATE

Product requirements

PRD.md / What should the product do, and why?

Problem → **Audience** → **Scope** → **Acceptance**

Product intent belongs here. Technology choices belong in the technical document.

Essential section	Questions to answer
Product summary	What are we building, for whom, and what is the smallest useful first release?
Problem and evidence	What happens today? What difficulty exists? Is the supporting information observed, reported, or assumed?
Audience and roles	Who is the primary user? What task matters to them?
Outcomes	What should improve? How will we measure it, and when will we review it?
Scope and priority	What is essential now, what is later, and what is explicitly excluded?
Requirements	What observable behavior must exist? What result would count as correct?
Constraints and questions	What limits are confirmed? Which unresolved question blocks which feature?
Release acceptance	Which named conditions and checks must pass before this release?

Separate success from acceptance
A portfolio can work correctly without yet producing more inquiries. Business outcomes need their own evidence. Passing route tests establishes functional behavior, not product impact.

Conditional: add business rules when eligibility, limits, calculations or ownership need explanation. Record the rule, an example and any exception.

Short fill-in starter / PRD.md
User [who] / Problem [what] / First release [scope] / Requirement R-[number] [behavior] / Acceptance [observable result].

Write testable requirements

Hypothetical examples for two separate projects.

Use a small requirement record

Field	What to enter
Requirement	R-[number], user need, required behavior and priority.
Acceptance	Given [starting condition], when [action], then [observable result].
Trace	Evidence/source [reference], flow F-[number], task T-[number], checks V-[numbers].

Example A / Personal project portfolio

Assumption: public project content, no accounts or user-data collection.
R-001: visitors can open a case study and return to the project list.

- **AC-001:** selecting a project displays its matching case study and content.
- **AC-002:** its direct URL remains usable after refresh.
- **AC-003:** the visitor can return to the project list.

Example B / Private text records

Assumption: signed-in users create and retrieve simple private text records. Update/delete functionality is excluded.
R-002: users can create and retrieve their own records.

- **AC-004:** the owner creates a valid record and retrieves its saved content.
- **AC-005:** the saved record remains available after signing out and in.
- **AC-006:** another user cannot retrieve the private record.
- **AC-007:** a failed save preserves valid input for retry.
- **AC-008:** success appears only after persistence succeeds.

Before building this feature

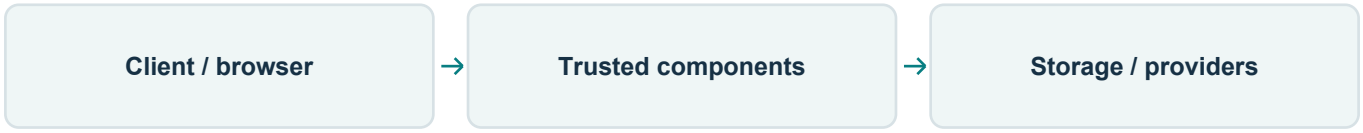
- Know the feature purpose, priority and acceptance conditions.
- Link release conditions to named verification checks. Keep results in the implementation plan.

Update when: audience, scope, priorities, business rules or acceptance change.

02 / TEMPLATE

Technical requirements

TECHNICAL_REQUIREMENTS.md / How will the system work?



Illustrative only. A static site may have no application service or user database.

Essential section	What to capture
Existing environment	Inspect the repository, tools, hosting and integrations. Identify working behavior to preserve.
Constraints	Supported environments, compatibility, deployment limits and confirmed cost constraints.
Architecture	Components, responsibilities, connections, trust boundaries and important data movement.
Technology choices	Choice, reason, alternative, trade-off and revisit trigger. Decide only what the next feature needs.
Quality requirements	Scenario, observable metric/result, justified target, representative conditions and check.
Security boundaries	Where identity, input handling, authorization and secrets are managed. Link detailed data permissions.
Development and operations	Setup, commands, variable names/purposes, deployment context and safe diagnostics.

Conditional: integration contracts, compatibility/migrations and significant architectural decisions.

Make quality observable
Replace "fast" with a named page or operation, a project-specific target, representative conditions and a measurement method. Do not invent numbers to fill the template.

Short fill-in starter / Technical requirements
Existing setup [facts] / Feature components [roles] / Key choice [reason] / Constraint [condition] / Check [method].

Keep technical choices clear

Record significant decisions; avoid paperwork for routine choices.

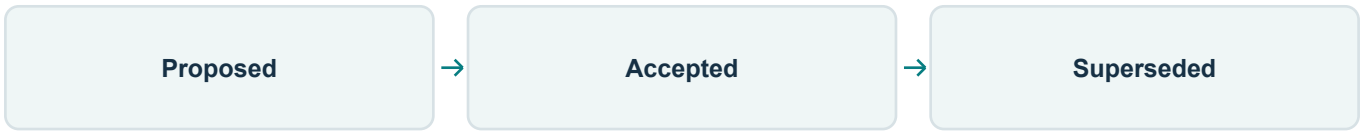
Integration record / Conditional

Interface [name] / Purpose [reason] / Inputs [contract] / Outputs [contract] / Authentication [method] / Limits and timeout [relevant values] / Failure and recovery [behavior] / Provider reference [version or URL].

Check the provider contract before dependent implementation. Define the event that establishes success. Provider acceptance may differ from the final outcome.

Significant decision record / Conditional

Record	Fields
ADR-[number]	Title; owner; status; accepted date when applicable.
Reasoning	Context; selected decision; alternatives; benefits, costs and risks.
History	Supersedes [ID or none]; superseded by [ID or none]; revisit trigger.



A proposal may instead be rejected. A later accepted record replaces an earlier decision.

Preserve an accepted decision's original rationale. Update lifecycle links when it is replaced; do not silently rewrite its history.

Examples / Same scope as the PRD

- **Portfolio R-001:** choose routing/content delivery that supports selection, direct URLs, refresh and return navigation. Check AC-001 to AC-003.

- **Records R-002:** trusted components establish identity, enforce ownership and confirm persistence. Failed saves enter the recovery flow. Check AC-004 to AC-008.

Before building this feature

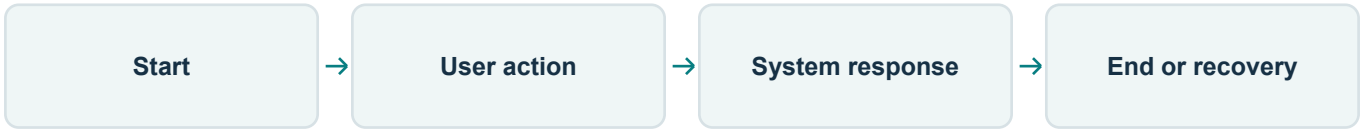
- Resolve the boundaries and contracts needed for this feature.

- Keep credentials out of documentation; store only safe configuration instructions.

Update when: architecture, tools, interfaces, constraints or operational responsibilities change.

App flow

APP_FLOW.md / How does a user complete the task?



A route map shows destinations. A flow also shows responses, decisions and failed paths.

Essential section	What to capture
Task inventory	Actor, goal, requirement and priority.
Destinations	Page, external link, download or action; its purpose, entry point and access restriction.
Navigation map	Page relationships, main navigation and contextual return paths.
Main flow	Actor, goal, preconditions, trigger, action/response steps and successful end state.
Alternatives and failure	Relevant invalid input, missing content, denied access, expired sessions, service failures and recovery.
Screen states	Applicable initial, loading, empty, populated, success and error states.
Interaction behavior	Mobile navigation, keyboard/focus transitions, back behavior, direct links and unsaved changes.

Do not redefine the same rule
The PRD owns what information is required and why. The flow owns when feedback appears and how users recover. The backend owns stored-data constraints. The design brief owns presentation.

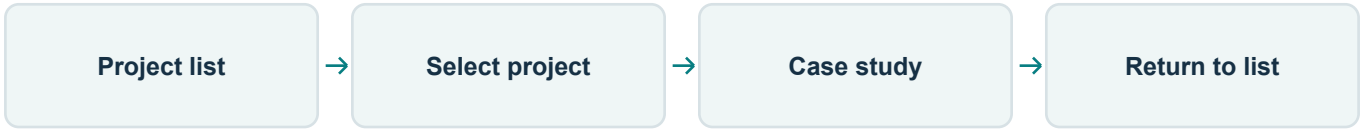
Conditional form record: product/data-rule references, validation timing, error feedback, input preservation, pending state, retry, cancellation, duplicate submission behavior and confirmation trigger.

Short fill-in starter / APP_FLOW.md
Flow F-[number] / Actor [who] / Start [condition] / Action and response [steps] / End [result] / Recovery [path].

Trace the whole journey

F-001 and F-002 connect to the same requirements in every document.

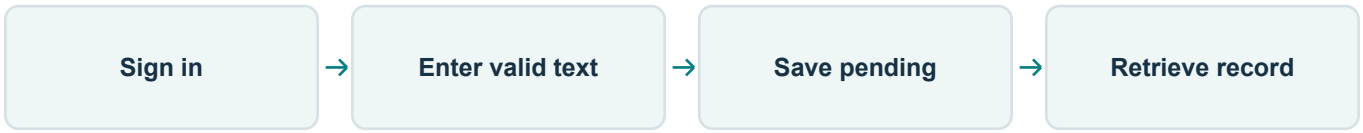
Example A / F-001: Explore a portfolio project



Public content; no account is assumed. R-001 / T-001 / V-001 to V-003.

Also specify direct-link and refresh behavior. Unknown project destinations need a useful not-found state. Visual and keyboard details link to DESIGN.md.

Example B / F-002: Save and retrieve a private record



Create/read only. Success follows persistence confirmation. R-002 / T-002.

Condition	Expected behavior
Save succeeds	Confirm success, then allow retrieval of the saved content.
Save fails	Keep valid input and provide the defined retry path. Do not show success.
User signs in again	The owner can retrieve the previously saved record.
Another user requests it	Deny retrieval without disclosing private content.

Acceptance: AC-004 to AC-008. **Checks:** V-004 to V-008. Field meaning and permissions are referenced from BACKEND_SCHEMA.md.

Before building this feature

- A builder can follow the main journey and relevant recovery without inventing behavior.
- Use a bounded prototype if the journey itself is uncertain.

Update when: tasks, routes, transitions, states, validation timing or recovery change.

Design brief

DESIGN.md / How should the interface look and respond?

Hierarchy → **Visual rules** → **UI components** → **Visible states**

These are design categories, not a prescribed palette, font or component library.

Essential section	What to capture
Intent	Audience, task, desired impression and what the user should notice first.
References	What qualities to use, why they help and which details are not prescribed.
Visual foundations	Color roles, type hierarchy, spacing, content width, borders and relevant icon treatment.
Responsive layout	Content order, navigation presentation and overflow/reflow as space changes.
Components and states	Purpose, variants, labels/help and relevant focus, pending, empty, success or failure presentation.
Content and assets	Voice, labels, errors, images, alternatives and ownership of required assets.
Accessibility and review	Relevant criteria, representative screens/conditions and manual/automated checks.

Classify consequential choices

Classification	Meaning
Required	Implement the specified decision.
Adaptable	Implementation may vary within stated constraints.
Exploratory	Validate the idea before adopting it.

Record: Decision [name] / Classification [type] / Specification or constraints [definition] / Rationale [reason]. Do not classify every tiny visual detail.

> **Short fill-in starter / DESIGN.md**
> Audience [who] / Main visual priority [what] / Required [decision] / Adaptable [constraint] / Screens and states [list].

Make design instructions usable

Clarify intent while leaving appropriate implementation freedom.

Reference modes

Mode	How to use it
Exact reference	Follow the specific aspects named in the brief.
Qualities to adapt	Reuse named qualities while fitting this product.
Inspiration only	Context, with no obligation to reproduce the image or page.

Example A / Personal project portfolio

Assumption: public project list and case studies.
Required: selected-project identity, readable case-study hierarchy and usable return navigation.
Adaptable: preview layout within responsive constraints.
Exploratory: optional animation, if it serves the task.

R-001 / F-001 / AC-001 to AC-003 / T-001. Add relevant visual and accessibility checks to the implementation plan.

Example B / Private text records

Pending → **Success OR failure** → **Clear next action**

The app flow controls when states occur. The design brief controls how they are presented.

Required: pending/failed persistence must be distinguishable from success. Errors must be understandable without color alone.
Adaptable: form layout within usability constraints.
R-002 / F-002 / AC-004 to AC-008 / T-002.

> **Accessibility is a target to verify**
> Use the project target; the starting default is WCAG 2.2 AA. Identify applicable criteria and checks. Automated scans or a filled template alone do not establish conformance. Specify motion alternatives when motion is used.

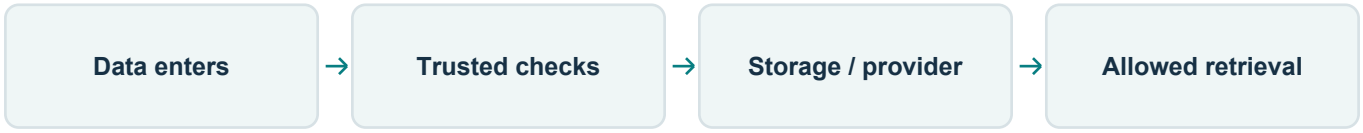
Before building this feature

- Define the feature hierarchy, key states and responsive behavior.
- Clarify consequential choices that are still exploratory.

Update when: visual rules, components, presentation, references or content conventions change.

Data and access

BACKEND_SCHEMA.md / What exists, and who can access it?



Illustrative only. Inventory real data handling even when there is no application database.

Section	What to capture
Data inventory - Core	Data category, purpose, storage/provider, retention and sensitivity. Include forms, analytics and browser storage if used.
Ownership - Core	Who owns the data, which source is authoritative and where its access policy is defined.
Entities/fields - Conditional	Meaning, logical/physical type, required/default values and who supplies or derives each value.
Integrity - Conditional	Relevant uniqueness, references, valid values, failed-operation behavior and parent/owner removal.
Permissions - Conditional	Actor, resource, operation, ownership/context condition and trusted enforcement point.
Queries - Conditional	Real access patterns, expected volume, pagination and justified indexing.
Lifecycle - Core	Collection, storage, retention, deletion responsibilities and external recipients.
Recovery - Conditional	Data migration, backup/restoration checks and irreversible-change limitations.

Two important boundaries
A hidden button does not enforce authorization. A client-supplied owner identifier does not establish trusted ownership. The technical document defines enforcement architecture; this document defines data permissions.

Short fill-in starter / Data and access
Data [what] / Location [where] / Owner [who] / Allowed access [rule] / Retention [rule or decision needed].

Specify only the data you need

Do not create a database or account system just to fill the template.

Example A / Personal project portfolio

Assumption: public descriptions/images; no accounts, forms, analytics or persistent browser storage.
Inventory: published project content.
Application database: not applicable. Keep a short data-handling statement and identify the authoritative content source.

R-001 / F-001 / AC-001 to AC-003 / T-001. Reassess this statement if collection or external services are introduced.

Example B / Private text records

Logical field	Meaning / trusted source
id	Record identifier, assigned by the trusted persistence layer.
owner_id	Owning user, derived from validated identity.
content	Saved text, with validity linked to the PRD.

Logical types are identifier, user identifier and text. Physical types and storage technology are intentionally unspecified.

Actor / operation	Condition
Signed-in user / create	The new record belongs to that validated user.
Owner / retrieve	The record belongs to the requesting user.
Another user / retrieve	Denied without private content disclosure.

R-002 / F-002 / AC-004 to AC-008 / T-002. No update/delete functionality is assumed. Resolve actual retention and owner-removal responsibilities before relevant production data is collected.

Before building this feature

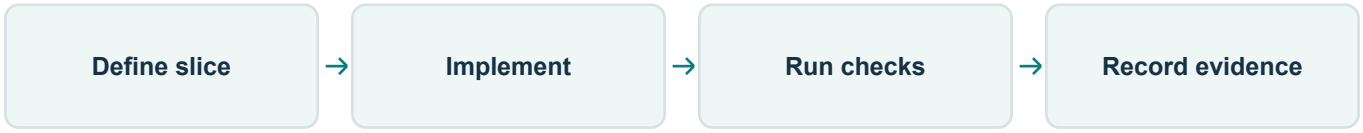
- Know field meaning, trusted ownership and relevant constraints.
- Define positive/negative checks; results belong in the implementation plan.

Update when: data, permissions, queries, retention or external handling change.

06 / TEMPLATE

Implementation plan

IMPLEMENTATION_PLAN.md / Build a small feature and verify it.



Implemented means the change exists. Verified means applicable required checks passed.

Keep the plan small

Essential record	What to enter
Milestone	User-visible result, requirement references and exclusions.
Dependencies	What the task needs, owner, resolution criterion and decision/source.
Implementation slice	A small end-to-end capability, requirements and task references.
Task definition	Goal, acceptance references, document sections, allowed scope, exclusions, dependencies and required checks.
Release/recovery	Candidate revision, acceptance contract, named checks, deployment, smoke checks and recovery trigger/procedure.
Agent handoff	Task reference and relevant document sections; link current progress instead of copying it.

Conditional exploration task: question [uncertainty] / bounded experiment [activity] / decision criterion [result needed]. It uses the same task-status table.

One authoritative task-status table

Task ID	Status	Blocker	Next action	Checks
[T-###]	Pending	[None or issue]	[Action]	[V-###]

Task statuses: Pending, Active, Blocked, Implemented, Verified. Keep current status, blocker and next action only in this table. Task descriptions define work; verification records hold check results.

When a decision is missing
Identify its owning document and resolve consequential uncertainty before dependent implementation. Continue unaffected work. Never invent an unperformed test result.

Short fill-in starter / Implementation plan
Task T-[number] / Goal [outcome] / Acceptance [reference] / Scope [boundary] / Checks V-[numbers] / Current progress [one table].

Check the right build

Reproducible evidence connects an acceptance condition to an observed result.

Verification field	What to enter
Check and expectation	V-[number]; requirement/acceptance reference; scenario; expected result.
Build and environment	Exact revision/build; environment; browser/device, viewport or other relevant conditions.
Execution and result	Method/command; observed result; Not run, Passed, Failed or Blocked.
Evidence	Location; checked-at time; coverage limits and unresolved uncertainty.

Worked check specifications / No results claimed

Example	Acceptance to check mapping
Portfolio / T-001	V-001 > AC-001: correct case study. V-002 > AC-002: direct link and refresh. V-003 > AC-003: return to list.
Records / T-002	V-004 > AC-004: owner creates/retrieves. V-005 > AC-005: persists across sessions. V-006 > AC-006: other-user retrieval denied. V-007 > AC-007: failed save preserves input. V-008 > AC-008: success only after persistence.

- Visual evidence names the screen, state, viewport, environment and build. Automated evidence names the command/suite, revision and result.

- Earlier evidence does not automatically verify changed behavior. Identify affected checks and rerun them when appropriate.

- Name the release-blocking checks. Record remaining issue, impact, whether it blocks release and the explicit decision. An exception does not turn a failure into a pass.

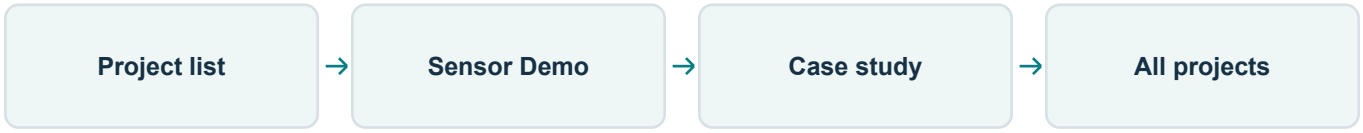
> **Ready to explore / Ready to implement / Ready to verify**
> Explore when a question and bounded experiment are clear. Implement when necessary behavior, data and technical decisions are resolved. Mark Verified only after applicable required checks pass and evidence is recorded.

Update: scope, dependencies, progress, blockers, evidence applicability or release conditions. This edition simplifies the previous templates without adding project facts. JEV was not rerun; all worked checks remain unexecuted examples.

CONNECTED EXAMPLE / 1 OF 2

One feature, six documents

A hypothetical portfolio example. No product checks have been run.



Illustrative routes: /projects/ and /projects/sensor-demo/. These are example choices, not user project facts.

Assumptions: static public content; no accounts, forms, analytics or persistent browser storage. Keep the existing suitable publishing setup.

Document	Concrete decision for this example
PRD.md / R-001	A visitor opens the listed case study and returns. AC-001: matching title, description and image. AC-002: direct URL/refresh works. AC-003: return to list.
Technical requirements	The host serves the case-study route directly and after refresh. Test that capability before dependent work if unknown. No new framework or database is prescribed.
APP_FLOW.md / F-001	Open list; select Sensor Demo; display the matching case study; select All projects; return to the list. Also support direct entry and refresh.
DESIGN.md	Required: clear project identity, readable content and visible keyboard-operable return link. Adaptable: layout within those constraints. No palette is prescribed.
Data and access	Public project text/images are published site content. No application database or visitor-data collection under these assumptions. Revisit if collection is added.
Implementation / T-001	Build that complete journey within the existing setup. Scope: content, listing link, case-study route and return link. Define V-001, V-002, V-003 and V-009.

A small example, not a larger app
Do not infer accounts, forms or persistence from the existence of six documents. The editable WORKED_EXAMPLE.md contains the same walkthrough and its unexecuted task/check records.

From acceptance to evidence

All checks below are specifications, not results.

Check	Action and expected result	Status
V-001 / AC-001	Select Sensor Demo from the list. Matching title, description and image appear.	Not run
V-002 / AC-002	Open the direct case-study URL, then refresh. The same content appears after both actions.	Not run
V-003 / AC-003	Select All projects. The /projects/ destination displays the project list.	Not run
V-009 / Design	Review at illustrative 375 and 1440 CSS-pixel widths and navigate by keyboard. Required content is not clipped; controls work and focus is visible.	Not run

The two widths are example review conditions, not universal breakpoints. This review is not a complete accessibility conformance evaluation.

Fill this only after an actual check

Evidence field	Current example value
Revision/build and environment	Not run
Method and relevant conditions	[Procedure, browser/device, viewport, content or test data]
Observed result and check time	Not run
Evidence location	None
Limitations	[Actual coverage and remaining uncertainty]

> **If the scope changes**
> A future contact form needs its own requirement and relevant integration, flow, design and data-handling decisions. The old "no visitor data collection" assumption must be reassessed. This illustration does not authorize adding a form.

Release rule: use named required checks against the candidate build. Keep task progress in one table. Preserve failed and Not run results honestly. Update affected documents and evidence when behavior changes.